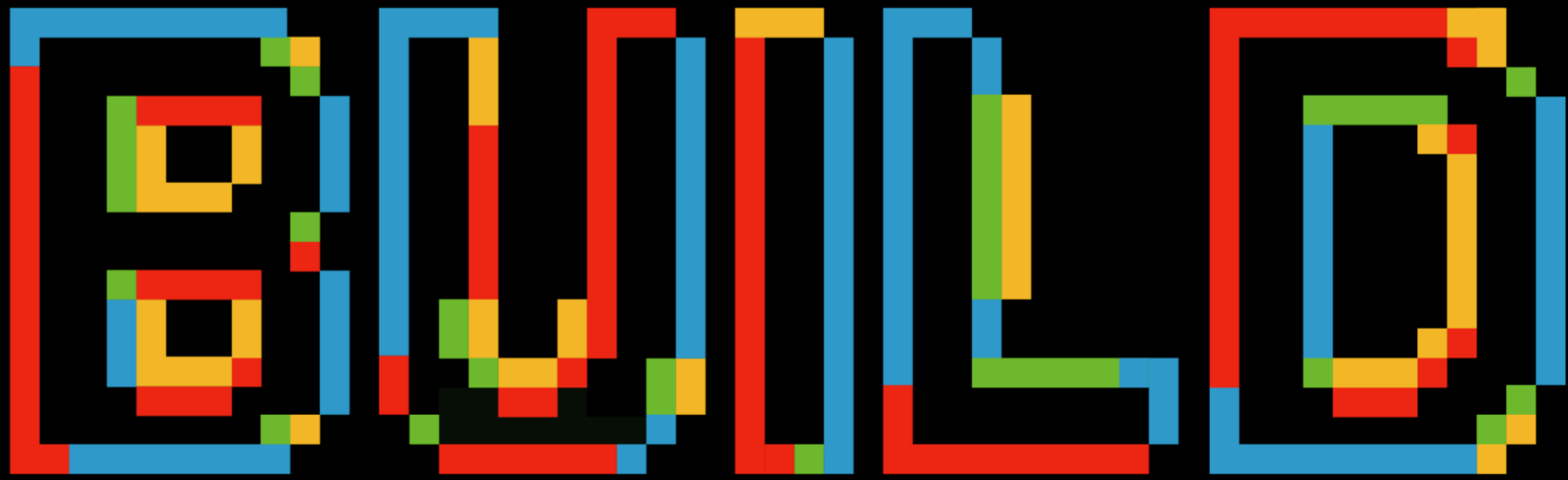
Microsoft Build //localhost:newyork

6/9/2026

8:00-9:00		Check In & Breakfast
9:00-9:05		Kickoff & Welcome
9:05-9:40	Peter Ward	*Keynote*: The hidden value of asking better questions with AI projects
9:40-10:15	Dave Goad	Moving Beyond Vibe Coding to Spec Driven AI Native Development in the Enterprise
10:15:10:35		Break
10:35-11:10	Manpreet Singh	Toolkit for building Agents: M365 Copilot, Studio & SharePoint in Action
11:10-11:45	Vladimir Gusarov	Your Backlog Is Lying: Enforce It with PR Checks
11:45-12:20	Preeti Gupta	Data Enablement in Regulated Finance: Moving Beyond Access to Outcomes
12:20-12:40		Break
12:40-13:15	Peter Smulovics	From UDDI to MCP – How Did Finding APIs Change in the Last 30 Years
13:15-13:50	David Patrick	From Zero to ChatGPT: Building Your Own AI Assistant with Azure AI
13:50-14:25	Monika Mundra	Observability Deep Dive: Correlating logs across azure monitor, app insights and distributed service
14:25-15:25		Lunch
15:25-16:00	Jason Beres	*Sponsor Presentation*: Building Enterprise Apps with AI, MCP and Components
16:00-16:35	Abhijeet Jadhav	Copilot Studio to Azure AI Foundry: A Framework for Knowing When — and How — to Scale
16:35-17:00		MVP Panel
17:00-		Networking / Mingle

Microsoft

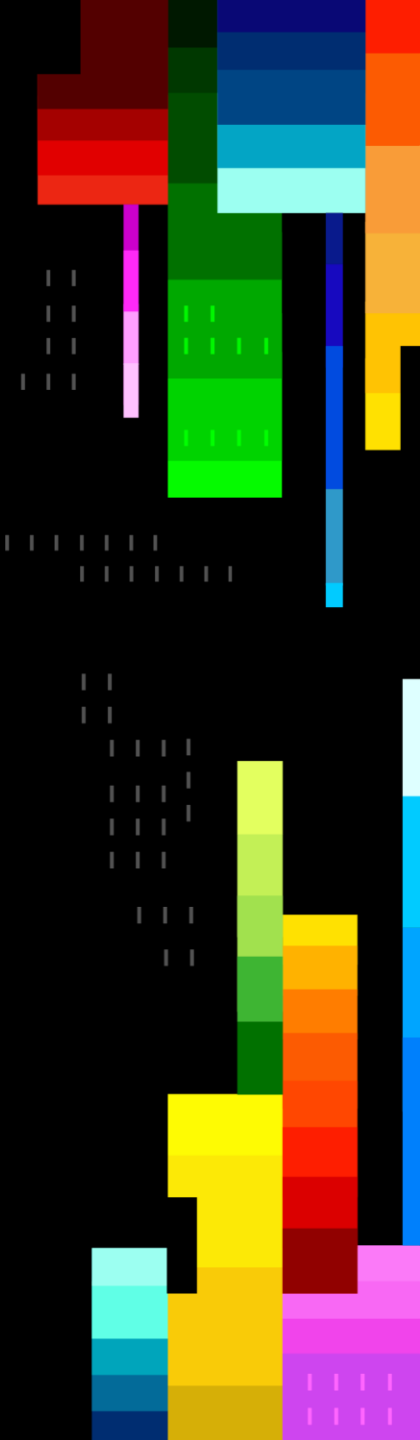


MICROSOFT BUILD

Microsoft

Your Backlog Is Lying: Enforce It with PR Checks

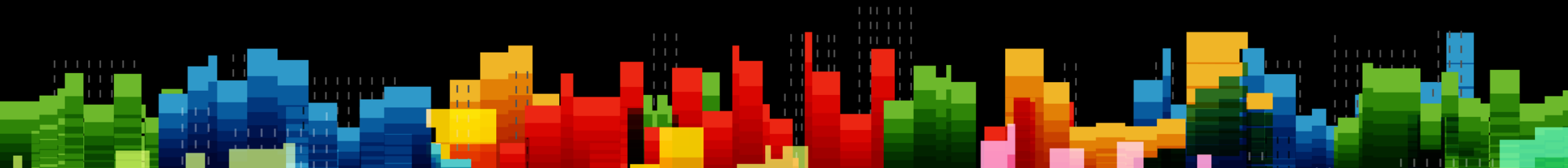
Vladimir Gusarov
Director of Software Engineering, Semperis
Microsoft MVP, Development Tools/DevOps



Agenda

- Backlog Consistency & Why It Matters
- Enforcing Consistent Standards
- Automated Backlog Validation
- Wrap-Up

Backlog Consistency & Why It Matters



The Power of a Well-Maintained Backlog

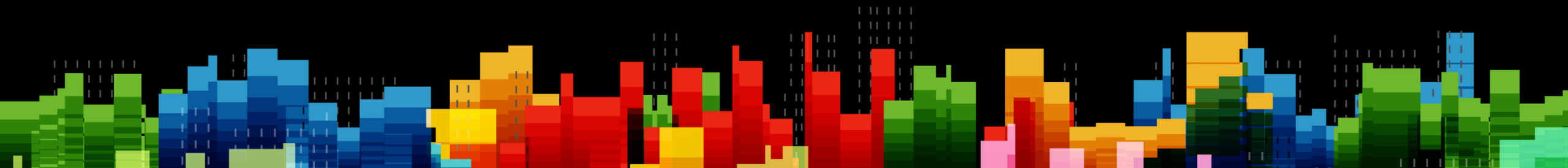
Predict implementation dates – Azure DevOps uses actual team velocity to forecast when backlog items will be delivered

Roll-up progress visibility – See real-time completion status across features and epics without chasing individual contributors

Metrics without the overhead – Get actionable planning data from the backlog itself, with less disruption to developers

Traceability for compliance – Every code change traces back to a planned, business-owner-approved work item, making it easy to audit why specific changes were made

Enforcing Consistent Standards



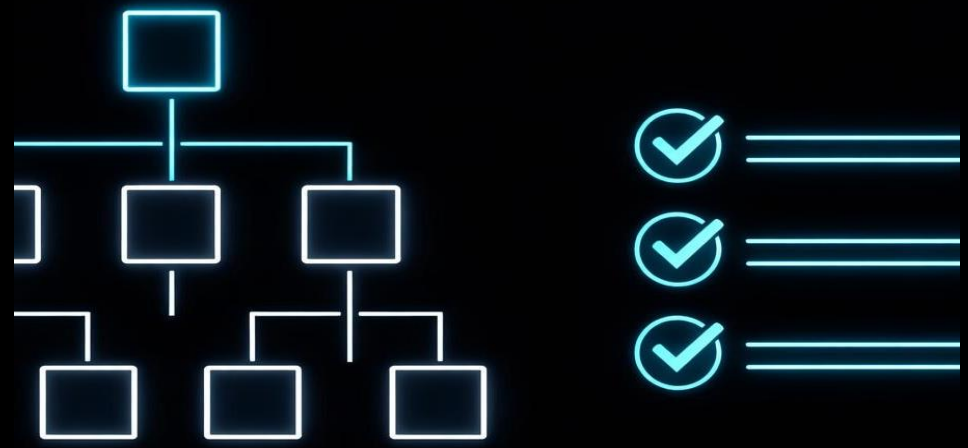
What It Takes to Make It Work

These capabilities only work when the backlog follows a consistent structure:

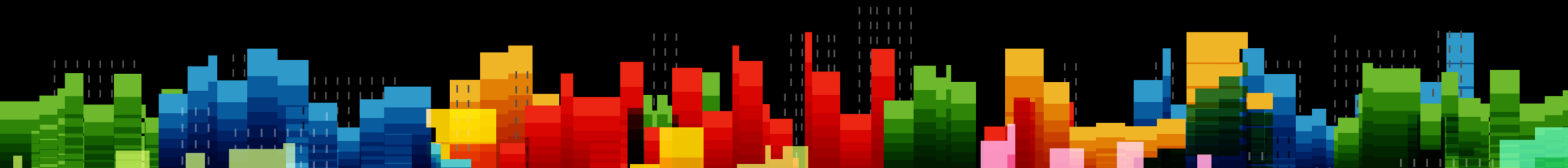
Follow the work item hierarchy – Epics → Features → Stories → Tasks must be linked correctly for roll-ups to work

Maintain estimates – Story points or effort fields need to be filled in so velocity calculations are meaningful

Enforce consistency at scale – Manual reviews don’t scale; you need automated checks to keep the backlog trustworthy



Automated Backlog Validation



Why Manual Reviews Fall Short

Inconsistency creeps in – Teams skip fields, mislink items, or forget estimates under deadline pressure

Reviews don’t scale – A person checking every work item is slow, error-prone, and unsustainable as the team grows

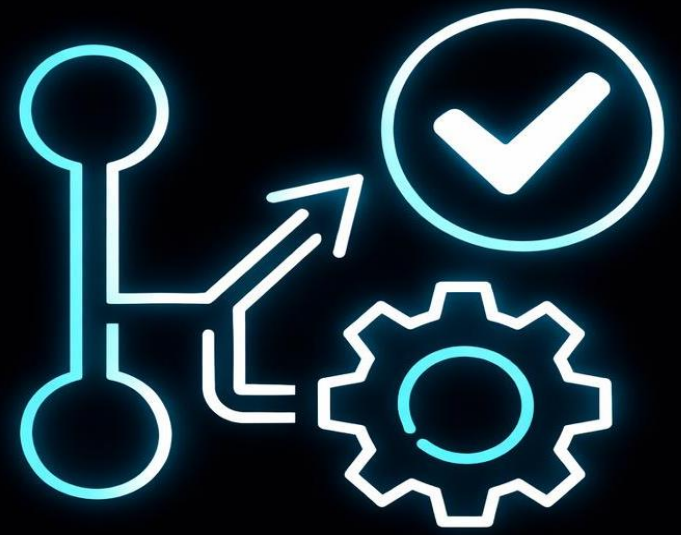
Problems surface too late – Broken metrics and bad forecasts are discovered at sprint review, not when the issue was introduced

PR Checks as Validation Gates

Shift validation left – Validate backlog compliance at pull request time, before code merges into the main branch

Automated and consistent – Every PR runs the same checks, removing human bias and ensuring no work item slips through incomplete

Developer-friendly feedback – Clear pass/fail status in the PR tells developers exactly what to fix before merging



Work Item Queries as Policy Rules

Define rules as queries – Write Azure DevOps work item queries that express your backlog policies (e.g., “all stories must have estimates”)

Flexible and extensible – Add new validation rules without changing code – just create another query and attach it to the PR check

Actionable results – Query results identify the exact non-compliant items, so developers know precisely what to fix

How to Enforce: Two Approaches

Option 1 – CI/CD with Work Item Query API

Query API in PR builds – Run a saved work item query in the CI/CD pipeline on each pull request

Validate linked work items – Fail the build when linked items are non-compliant

Option 2 – Specialty software like PolicyVault.io

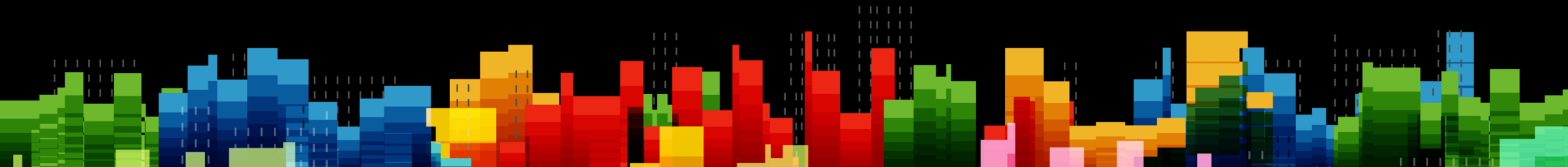
One place – manage validation rules centrally with flexible scoping across projects

Per-team flexibility – Different policies per team or project, no duplicated pipeline code

Demo

Enforce Policies Based on Work Items Queries

Wrap-Up

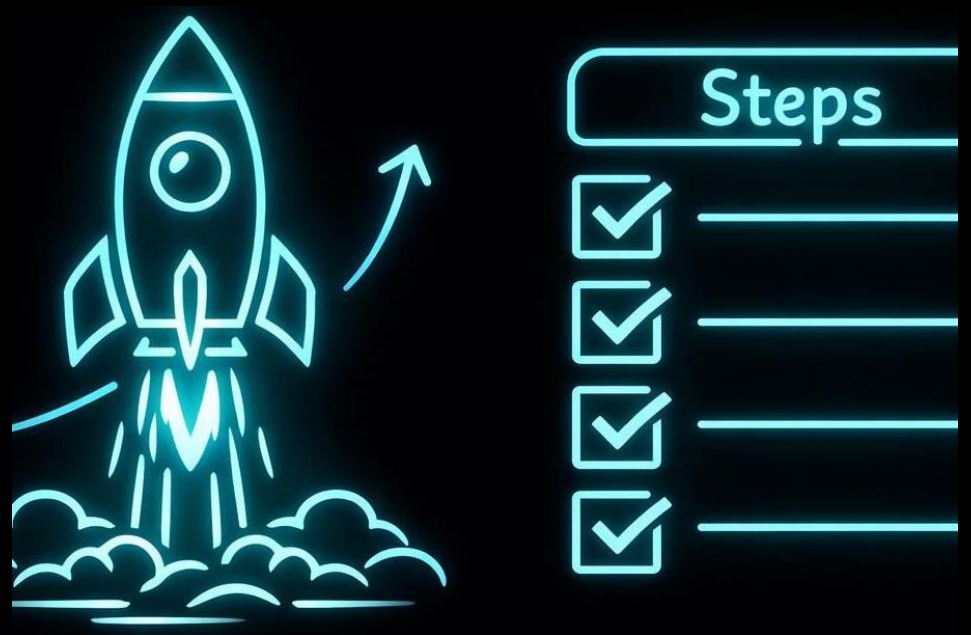


Getting Started

Write your validation queries – Start with one or two rules that match your most common backlog gaps

Add a PR build step – Wire up the query API call in your existing CI/CD pipeline as a required check

Iterate and expand – Add new policies over time as the team identifies more consistency gaps



Key takeaways

- Backlog can give you a forecast and roll-up progress only when has consistent structure
- Enforcing consistent standards across work items makes planning and reporting metrics reliable
- PR checks can automate backlog validation, catching issues before they enter the pipeline
- Policy-based enforcement with work item queries shifts validation left and reduces manual overhead

Q&A

Vladimir Gusarov

E-mail: vg@almguru.com

GitHub: @almguru